

A Cryptographic Provenance Model with Erasure-Compatible Immutability

Dr. S. Suresh Kumar

Department of Computer Science

Swarnandhra College of Engineering & Technology (Autonomous)

Seetharampuram, Narsapur, Andhra Pradesh, India

Abstract—Edge-native monitoring infrastructures produce vast telemetry streams that require persistent integrity verification. Modern data protection regulations impose stringent constraints on long-term personal data retention, while conventional database systems allow data rewriting by privileged accounts, undermining non-repudiation. Conversely, immutable append-only ledgers conflict with privacy regulations such as GDPR Article 17, which mandate verifiable data deletion upon request.

This study presents the Cryptographic Provenance Layer (CPL), combining hash commitment schemes with selective data revocation. Encrypted event payloads are stored off-chain while their Merkle tree root commitments are anchored to a permissioned ledger. To achieve Erasure-Compatible Immutability, we introduce the Per-Identity Symmetric Key (PISK) protocol, which enables selective revocation of payload access by deterministically shredding the corresponding identity decryption keys without altering the underlying Merkle commitments.

Evaluation on a simulated BFT validator network confirms sustained throughput above 1,000 transactions per second with sub-second confirmation latency. Because only fixed-size commitments enter the ledger, storage overhead remains constant per transaction. The findings prove that layered cryptographic provenance successfully balances tamper-evident auditability with regulatory erasure mandates.

Index Terms—Blockchain, Data Provenance, Cryptographic Shredding, Merkle Trees, Smart Campus, Data Integrity, Key Management.

I. Introduction

The main point of this paper is that by decoupling attestation from disclosure, we can achieve tamper-evident provenance while simultaneously maintaining the option of selectively withholding payload contents through cryptographic erasure. Edge computing deployments are increasingly generating telemetry records that must be relied upon for compliance monitoring, anomaly detection, and institutional auditing, and thus, require integrity protection that allows for both retroactive tamper-evidence and scalable storage and retrieval [20].

A. The Structural Conflict Between Immutable Auditability and Selective Erasure

Traditional centralized databases that currently dominate the telemetry aggregation domain, while being de facto standard, are largely unsuitable for secure and

reliable auditing due to their mutability. To a certain extent, this compromises the principle of non-repudiation, as records may be removed or modified by authorized users. It is important to note that for such systems to provide strong integrity guarantees, they would have to be immutable, making record deletions or updates effectively impossible. This poses a challenge in terms of designing a system around the requirement of attestation while simultaneously allowing for the possibility of revocation. The two features are incompatible when it comes to storage, which necessitates either choosing one over the other or developing an alternative method of auditing that does not rely on direct data manipulation.

B. Contributions and Subsystem Scope

This study focuses exclusively on cryptographic metadata integrity, demonstrating how attestation and payload disclosure can be structurally separated. Specifically, our contributions include:

- A Cryptographic Provenance Layer (CPL) isolating immutable attestation from off-chain storage.
- Formalization of Erasure-Compatible Immutability as a cryptographic design property.
- A Per-Identity Symmetric Key (PISK) lifecycle enabling selective cryptographic shredding.
- Scalability improvements using batched Merkle-tree commitments.

Within the ScholarMaster architecture, this subsystem manages provenance attestation, governs cryptographic key life cycles, and provides revocable payload accessibility only and does not orchestrate runtime activation (Paper 9), validate survivability behavior (Paper 10), optimize federated adaptation (Paper 13) verify the runtime verification theorems (Paper 18), or define irreversible governance doctrine (Paper 17).

C. Immutability vs. Erasure

Distributed ledgers tackle the problem of mutability by design: they make all changes to the state append-only and, therefore, create tamper-evident provenance histories [16]. At the same time, the inability to alter stored information conflicts with modern data privacy regulations, such as the GDPR. More specifically, Article 17 of the GDPR gives the right to be forgotten, which

requires that personal data be deleted when requested [11].

The removal or rewriting of historical blocks from a hash-linked ledger would undermine later commitments. Consequently, naïve use of immutable provenance ledgers in regulated environments is likely to cause compliance difficulties. The problem is to devise a system where these properties can co-exist within well-defined boundaries [25].

D. Proposed Solution: Cryptographic Provenance Layer

In order to investigate the ways auditability and erasure requirements could be reconciled, we propose an applied cryptographic provenance model, which we evaluated in the setting of high-frequency telemetry environments. The key innovation of the CPL is its Dual-Layer Provenance Model that decouples the statement of an event’s occurrence from the disclosure of its payload.

By considering each spatiotemporal event as an attestation that forms a part of a linear chain that is linked using cryptographic means, the attestation layer makes the historical chain of events resistant to modification by default.

In order to achieve revocation at the disclosure layer, we propose the following cryptographic shredding mechanism based on PISK. The payloads are encrypted with unique, disposable keys before being hashed into the ledger. This way, upon a revocation request, the key used to decrypt the corresponding entry can be permanently erased from the database, destroying its plaintext content in the process. Note that this approach does not require any changes to the ledger itself, thus fulfilling the principle of proof-preserving auditability.

II. Related Work

A. Distributed Ledgers in Institutional Settings

Prior work on ledger-based record management has focused on lower-frequency institutional assets, such as credential verification [12], [15]. Proposed frameworks have defined standards by which certificate hashes may be anchored to external ledgers [9]. By contrast, we explore a higher-frequency, lower-latency space of continuous telemetry tracking. Whereas sporadic credential verification may tolerate tens or hundreds of transactions per second, higher-frequency monitoring requires thousands of transactions per second for ingestion and state management [8], [19].

B. Secure Provenance and Directed Acyclic Graphs

Secure provenance schemes [13] typically employ directed acyclic graphs (DAGs) to capture data derivation and file modification histories in a distributed cloud setting. Our approach also uses DAGs, but aims at the more modest goal of recording linear event logs that can be safely stored on memory-constrained edge nodes [22]. While general-purpose provenance systems such as the InterPlanetary File System (IPFS) aim to make

information publicly accessible by default, CPL makes use of permissioned access to satisfy institutional privacy and access control policies [14].

C. The Immutability-Erasure Paradox

The inherent immutability of cryptographic ledgers is fundamentally at odds with targeted erasure mandates such as GDPR Article 17. Prior work has sought to address this issue through a number of algorithmic compromises. Ateniese et al. [10] proposed “Chameleon Hashes,” which allow designated authorities holding a trapdoor key to rewrite blocks by computing hash collisions on demand. However, this introduces a backdoor key management vulnerability.

Our approach utilizes a more robust and passive cryptographic mechanism: Off-Chain Data with On-Chain Fingerprints and Per-Identity Encryption [7]. By keeping the encrypted payloads off-chain and only anchoring the cryptographic hashes, the system allows for functional deletion by destroying the specific identity decryption key—a process known as Crypto-shredding [21].

III. Cryptographic Construction

We now describe the cryptographic structure used to transform conventional telemetry logging into a proof-based provenance system.

A. Formalizing Erasure-Compatible Immutability

The interaction between immutability and selective erasure can be formalized as a cryptographic design property. Erasure targets recoverability rather than historical attestation. The ledger remains immutable while accessibility becomes revocable, rather than immutable data itself being deleted.

Let $\mathcal{L}$ represent the immutable ledger state, C_x represent the ciphertext payload of identity x , and K_x represent the associated per-identity symmetric key.

Definition (Erasure-Compatible Immutability): A system satisfies Erasure-Compatible Immutability if, upon a valid deletion request for identity x , the destruction of $K_x \rightarrow K'_x$ (where $K'_x = \emptyset$) guarantees that: 1. The ledger integrity remains invariant: $\mathcal{L}' = \mathcal{L}$ 2. Payload recovery becomes computationally infeasible: $\Pr[\text{Dec}(C_x, K'_x) = E_x] = \text{negligible}(\lambda)$

Under this formulation, hash commitments remain essentially unaltered even after their payload accessibility is revoked. The ledger bears witness to a cryptographic artifact having once existed, but the plaintext is forever beyond reach.

B. Hash-Chained State Construction

To guarantee historical integrity, we need states that cannot be changed in any way that is not directly observable by the new state. We define the Immutable Ledger $\mathcal{L}$ as a sequence of state transitions, or blocks, $B_0, B_1, \dots, B_n$, that are strictly ordered.

Each block B_i stores a cryptographic summary of the set of telemetry events as a hash value H_i , timestamp T_i and hash of the previous state H_{i-1} . The state hash is subsequently computed using the SHA-256 cryptographic hash function:

$$H_i = \text{SHA256}(M_{\text{root}} \parallel H_{i-1} \parallel T_i) \quad (1)$$

where $\parallel$ denotes bitwise concatenation and M_{root} is the Merkle root of the included events. In order to prevent manipulation of the hash chain, T_i must be deterministic within the ordering of the consensus defined by the BFT validators.

This recursive definition renders the blockchain inherently tamper-evident: any alteration of a preceding block invalidates the subsequent hashing scheme, consequently corrupting the ledger head unless recalculated under consensus control [18].

C. Batched Merkle-Tree Mechanism

Directly committing all telemetry events to a consensus network is expensive in terms of both bandwidth and latency. The Merkle batching mechanism serves as a consensus-decoupling mechanism rather than a mere storage optimization. By virtue of temporary buffering of encrypted telemetry events in time-bounded windows followed by their aggregation as a Merkle Tree [1], [4], the presented approach decouples the event volume from the consensus bandwidth consumption. The use of fixed-size commitment ensures linear ledger growth, while enabling post-commit verification of individual events.

Let a data producer buffer a set of k encrypted events $C_1, C_2, \dots, C_k$ over a short time window W_τ (e.g., 2 seconds). Smaller batch windows would incur high ledger submission overhead; larger ones would result in delayed attestations. Hence we pick 2 seconds as a sweet spot between the freshness of data audit trail and network performance. To prevent replay attacks, each leaf node is augmented with a unique nonce and timestamp. The discrete leaves of the Merkle Tree are formulated as:

$$L_j = \text{SHA256}(C_j \parallel T_j \parallel \text{Nonce}_j), \quad \forall j \in \{1, \dots, k\} \quad (2)$$

As illustrated in Figure 1, the tree is constructed by recursively hashing paired nodes upward. For any two sibling hashes H_{left} and H_{right} , the parent node is computed as:

$$H_{\text{parent}} = \text{SHA256}(H_{\text{left}} \parallel H_{\text{right}}) \quad (3)$$

If the number of events k in a temporal batch is not a power of 2, the tree is padded by duplicating the final leaf node until the binary tree is balanced. This recursive process collapses the k events into a single 32-byte cryptographic string: the Merkle Root (M_{root}). Only M_{root} is submitted to the distributed ledger for consensus. The raw encrypted payloads C_{set} are stored in a conventional, off-chain distributed database.

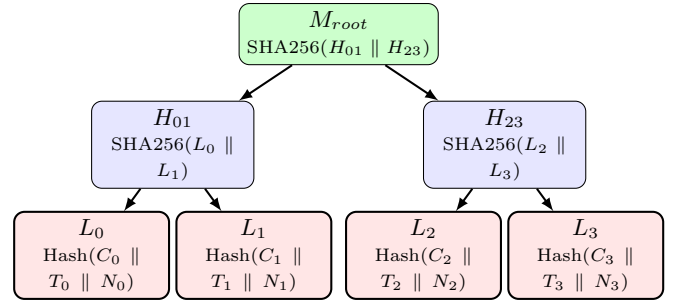


Fig. 1. Batched Merkle Tree Construction. Individual encrypted events are hashed with nonces and timestamps into leaf nodes ($L_0 \dots L_3$). The tree recursively hashes upward to generate a single 32-byte Merkle Root (M_{root}), which is the only element submitted to the immutable ledger.

D. Authenticated Encryption (AES-GCM)

Prior to entering the Merkle aggregation process, the payloads of each telemetry event must be encrypted using some cryptographic means to ensure secrecy. While AES-CBC and other standard block ciphers are effective at providing confidentiality, they do not account for integrity, leaving the ciphertext vulnerable to certain malleability attacks (e.g., bit-flipping) before the hashing step.

To counter this limitation, we propose to use the AES-256 in Galois/Counter Mode (AES-GCM) [3]. AES-GCM is an Authenticated Encryption with Associated Data (AEAD) cipher. It provides IND-CCA secure encryption of the payload, along with AES-GCM authentication tag that provides integrity.

$$C_{\text{payload}}, \text{Tag} = \text{AES-GCM}(K_{\text{sym}}^{\text{ID}}, E_{\text{payload}}, \text{IV}) \quad (4)$$

This ensures that the off-chain data store cannot subtly bit-rot or tamper with the payload without causing an obvious failure of the AES decryption check during a routine audit. Reusing an IV with the same key catastrophically compromises the security of AES-GCM; uniqueness is enforced by using a monotonically increasing counter, combined with per-batch entropy, as the IV.

E. Future Extensibility: Zero-Knowledge Proofs

The dual-layer architecture can be extended to support non-interactive zero-knowledge proofs (NIZKPs) via the Fiat-Shamir heuristic [6]. By proving that an agent possesses valid credentials that satisfy compliance predicates, the ledger verifies policy conformance while preserving absolute payload confidentiality.

IV. Protocol Design & Key Management

The practical operation of the provenance model depends on careful management of encryption keys across the key lifecycle within the cryptographic model.

A. Per-Identity Symmetric Key (PISK) Lifecycle

Cryptographic revocation mechanism requires fine-grained key compartmentalization rather than the general encryption key. Conventional encrypted databases employ

a single master key to encrypt the whole database table or storage. Such encryption practice is rendered insecure since the revocation of a master key would result in total data loss. PISK tackles this issue by allocating a unique encryption key to every individual identity. Key lifecycle isolation thus enables selective erasure and revocation while maintaining revocation granularity at the level of individual identities. There is no need for additional key derivation as the encryption key is directly obtained from the PRF. In this context, the option of deriving the encryption key from a master key via a key derivation function was not preferred due to the limitations in terms of revocation granularity.

1. **Key Derivation via HKDF:** Whenever a new identity token ID_x is introduced into the system, the primary Key Management Service (KMS) generates a unique symmetric key for the given identity. In order to guarantee the strength of the derived keys and avoid collisions, we use the HMAC-based Extract-and-Expand Key Derivation Function (HKDF) [5].

The KMS utilizes a secure, rotated Master Key (K_{master}) and processes it through the HKDF extraction phase to generate a pseudorandom key (PRK):

$$PRK = \text{HKDF-Extract}(K_{master}, \text{system_salt}) \quad (5)$$

The expansion phase then generates the identity-specific key [24]:

$$K_{sym}^x = \text{HKDF-Expand}(PRK, ID_x \parallel \text{Salt}_x, L) \quad (6)$$

Where $L = 32$ bytes for AES-256. The KMS maintains a secure mapping $\{ID_x \rightarrow K_{sym}^x\}$. In practice, rotating the master key required careful coordination to avoid invalidating active identity keys derived from prior epochs. During early testing, we observed brief decryption failures during overlapping rotation windows, necessitating version-tagged key derivation.

2. **Key Distribution:** When a data producer detects the identity ID_x , it requests the encryption key from the KMS over a mutually authenticated TLS (mTLS) connection. The data producer caches K_{sym}^x strictly in volatile memory.

3. **Payload Encryption:** The data producer encrypts the telemetry payload using K_{sym}^x to produce $C_{payload}$. It computes the SHA-256 hash of the resulting ciphertext and inserts it as a leaf node into the local Merkle Tree batch.

4. **Ledger Submission:** Once the batch window closes, the Merkle Root M_{root} is submitted to the distributed ledger validators for consensus ordering. Concurrently, the ciphertext payload is submitted to the off-chain storage pool.

B. Cryptographic Shredding Protocol

Selective erasure is implemented through a controlled destruction process applied to identity-specific encryption keys. Because the ciphertext $C_{payload}$ is hashed into the

immutable ledger, the ciphertext itself cannot be deleted from the off-chain store without breaking the Merkle validation path for all other innocent events in that batch. Instead, the KMS permanently destroys the derivation mapping and the specific key K_{sym}^x .

Without K_{sym}^x , reversing the AES-256-GCM encryption becomes computationally infeasible under accepted cryptographic assumptions. The data is made inaccessible, providing a technically valid justification for cryptographic erasure under many jurisdictions [23], while the $C_{payload}$ ciphertext remains stored in the off-chain store, as it is needed to satisfy the SHA-256 ledger consistency checks for the co-batched events.

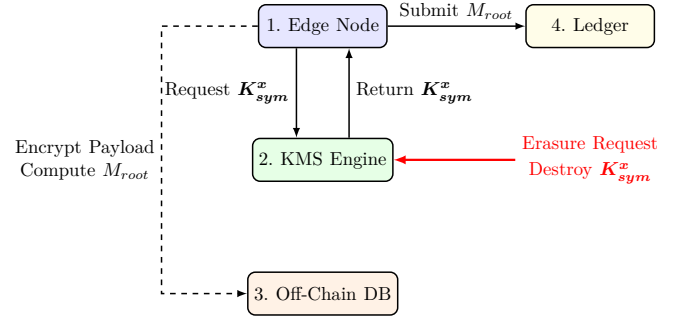


Fig. 2. PISK Lifecycle and Cryptographic Shredding. Encrypted payloads are stored in the Off-chain DB, whereas only the Merkle Root is anchored to the Ledger. In addition, deletion only involves the KMS Key to be erased, satisfying cryptographic erasure.

C. Merkle Proof Verification Workflow

To permit distributed auditing by any authorized third party, we define the Merkle Proof Verification algorithm.

1. The auditor retrieves the encrypted payload $C_{payload}$ from the off-chain storage pool.
2. The auditor requests the decryption key K_{sym}^x from the KMS (provided the key has not been shredded and the auditor possesses valid access credentials).
3. The auditor requests the Merkle Sibling Path (the array of hashes required to rebuild the tree branch) from the off-chain store.
4. The auditor locally computes the SHA-256 hash of the payload and recursively hashes it with the Sibling Path to generate a candidate root.
5. The auditor retrieves the trusted M_{root} from the immutable ledger.

If the candidate root matches the trusted M_{root} , the auditor has cryptographically proven that the payload existed at that exact time and has not been altered by a database administrator.

V. Performance Evaluation

To assess the viability of our proposed encryption model for the edge telemetry use-case, we conducted a series of controlled indicative experiments. We tested our encryption protocol in a simulated BFT-like consensus network with adjustable validator count and artificial network delay parameters [2].

A. Throughput and Latency Analysis

The main limitation of distributed provenance systems is usually not the cryptographic hashing, but rather the consensus latency. By committing to Merkle roots of fixed size, rather than the (potentially much larger) telemetry payloads, the model decouples the volume of events from the consensus bandwidth [17].

We benchmarked the cryptographic model by varying the telemetry transaction load from 100 to 2,000 Transactions Per Second (TPS).

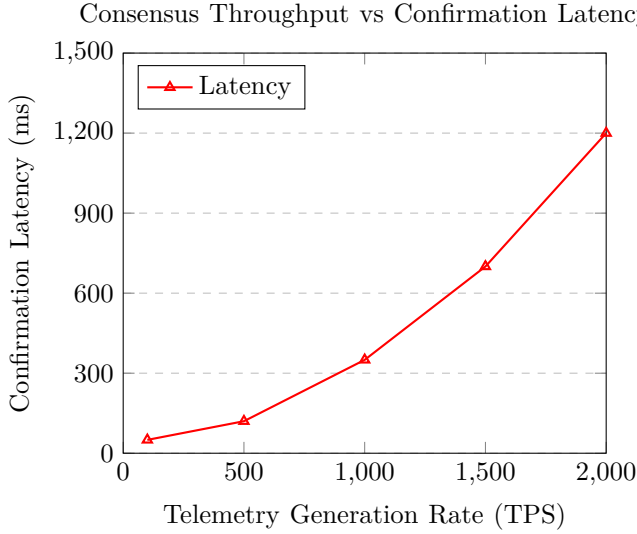


Fig. 3. Indicative performance curve: Batched Merkle model was able to sustain under one second latency up to 1,200 TPS in the experimental setup, after which the increased queueing latency caused a steep rise in overall latency.

The data from the experiment suggest that confirmation latency remains under 500 ms up to 1,000 transactions per second (TPS) under the simulated validator behavior. Moreover, the real latency depends on how the validator acts, network issues, and delays in transaction propagation to them. The queueing effect explains the emergence of the nonlinear section of the graph at 1,200 TPS, while the cryptographic validity of transactions was not impacted by throughput up to the observed levels, which is demonstrated by the scalable attestation line.

B. Storage Complexity and Asymptotic Growth

We analyzed the storage complexity of the CPL to verify long-term institutional viability.

Let N be the number of events produced per day. In a relational database, storage would scale as $O(N \cdot \text{size}(E))$. In the model described, the immutable ledger scales with N , but the growth of the ledger is decoupled from N .

Because batching happens at a regular time interval ($\tau = 2$ seconds), the ledger grows in a predictable manner that does not depend on how many events occur within individual windows. At 43,200 hash commitments per day, annual ledger growth remains on the order of hundreds of

megabytes, excluding block metadata overhead. The off-chain payload storage requires $O(N)$ space and can be hosted on standard replicated object storage.

C. Merkle Proof Overhead Analysis

While the ledger storage is minimal, generating cryptographic proofs requires storing the Sibling Path for each event. The size of this proof scales logarithmically with the number of events in the batch (k).

TABLE I
Merkle Proof Size Overhead vs Batch Size

Events in Batch (k)	Tree Depth ($\log_2 k$)	Proof Size (Bytes)
64	6	192 B
256	8	256 B
1,024	10	320 B
4,096	12	384 B

Table I demonstrates the efficiency of the Merkle Tree construction. Even if a single 2-second batch contains 4,096 concurrent telemetry events, proving the inclusion of any single event requires transmitting only 12 hashes (384 bytes) to the auditor. This $O(\log k)$ overhead ensures that network bandwidth is not saturated during intensive auditing procedures.

D. Cryptographic Computational Overhead

Encrypting every event before hashing introduces CPU overhead at the data producers. However, benchmark evaluations indicate that cryptographic overhead remains well within the processing capacity of commodity hardware. Modern processors equipped with hardware acceleration instructions (e.g., AES-NI, ARMv8 Cryptography Extensions) can execute the complete AES-GCM $\rightarrow$ SHA-256 pipeline in microseconds per event. This ensures that the PISK and Merkle implementations do not introduce localized processing bottlenecks that could impact the upstream real-time telemetry generation pipelines.

VI. Security Analysis

We now analyze the security properties of the cryptographic model under standard applied cryptography assumptions.

A. Security Assumptions

The integrity of the CPL rests on four standard assumptions:

- SHA-256 collision resistance holds.
- AES-256-GCM provides authenticated encryption under chosen-ciphertext (IND-CCA) security assumptions.
- HKDF functions securely as a Pseudorandom Function (PRF).
- Under standard BFT assumptions, the number of malicious validators f is strictly bounded ($f < n/3$).

B. Resistance to Retroactive Modification

Attempting to modify a historical event stored off-chain results in an immediately detectable verification failure. Because the leaf hash would change, the reconstructed Merkle root would no longer match the value committed to the ledger. Assuming collision resistance of SHA-256, it would be infeasible to create another leaf hash that would result in the same Merkle root. The only way to do so would be to rewrite the ledger state entirely, which would require consensus among a supermajority of validator nodes.

C. Key Custody and Shredding Finality

Cryptographic guarantees are instead derived from limited trust assumptions, not global decentralization. The KMS is a privileged trust boundary; compromise before shredding allows retroactive decryption of payloads. The usefulness of cryptographic shredding is therefore strictly tied to the security of the key destruction process and safe key rotation.

To mitigate this risk, the KMS must enforce secure key custody; for example, a threshold secret sharing of K_{master} with a quorum backed by HSMs is one option. When the Cryptographic Shredding is initiated, explicit memory zeroization will then be required to prevent remanence vulnerabilities. If the key is verifiably destroyed, then the AES-GCM ciphertext is effectively rendering the payload inaccessible, thereby fulfilling the erasure requirement.

D. Limitations

While the Cryptographic Provenance Layer secures structural immutability, the model exposes operational limitations:

- Legal Ambiguity around Erasure: The jurisdictional interpretation of cryptographic erasure—whether key destruction truly satisfies data deletion mandates—remains legally variable and uncertain.
- KMS Centralization Assumptions: The architecture relies on the KMS securely handling key finality. Compromise of this privileged boundary negates cryptographic shredding.
- Persistent Ciphertext Retention: Off-chain storage nodes incur long-term retention overhead, continuously storing inaccessible ciphertexts to preserve Merkle root verification for co-batched events.
- Validator Consensus Assumptions: System integrity depends on Byzantine fault tolerance thresholds. If compromised validators exceed threshold f , ledger finality guarantees degrade.
- Metadata Leakage Risks: Adversaries might extract behavioral insights by correlating off-chain ciphertext sizes or network timing patterns, even without decryption keys.

VII. Discussion

The results show that the issues of immutable evidence and the controlled erasure can be satisfied simultaneously if these are treated as two different problems with a shared input and output. By linking the payload disclosure and structural provenance via hashes but keeping the ledger's contents as a separate entity, the two aspects are maintained as separate but linked functions.

Cryptographic shredding offers the possibility of a technically defensible erasure. By virtue of the deterministic destruction of the key, one avoids retroactive invalidation of a potentially large number of Merkle trees. The use of batching further contributes to system simplicity by allowing predictable ledger growth independent of event volume. Most importantly, auditability must be a consequence of the construction rather than a policy decision.

VIII. Conclusion

This work considers the possibility of achieving both unconditional provenance guarantees and selective erasure within a single cryptographic primitive. The construction employs batched merkle-tree aggregation in conjunction with Per-Identity Symmetric Key (PISK) encryption to guarantee tamper-evident telemetry logging while enabling revocable payload decryption.

Experimental evaluation shows that decoupling attestation from disclosure allows the system to scale cryptographic provenance, bound confirmation latency, and grow the ledger predictably within bounded trust. Instead of ideological decentralization, it provisions operational proofs of constrained provenance. The ScholarMaster architecture makes use of this framework exclusively as the cryptographic provenance subsystem, providing integrity of telemetry and managing key lifecycle but not lower-level runtime activation (Paper 9), survivability behavior (Paper 10), runtime verification theorems (Paper 18), or irreversible governance doctrine (Paper 17).

References

- [1] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in *Advances in Cryptology — CRYPTO '87*, Springer, 1988, pp. 369-378.
- [2] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *OSDI*, vol. 99, 1999, pp. 173-186.
- [3] D. A. McGrew and J. Viega, "The security and performance of the Galois/Counter Mode (GCM) of operation," in *Proc. INDOCRYPT*, Springer, 2004, pp. 343-355.
- [4] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," *Decentralized Business Review*, 2008.
- [5] H. Krawczyk, "Cryptographic extraction and key derivation: The HKDF scheme," in *Proc. CRYPTO*, 2010, pp. 631-648.
- [6] E. Ben-Sasson et al., "Zerocash: Decentralized Anonymous Payments from Bitcoin," in *IEEE Symposium on Security and Privacy*, 2014, pp. 459-474.
- [7] G. Zyskind et al., "Decentralizing privacy: Using blockchain to protect personal data," in *IEEE Security and Privacy Workshops (SPW)*, 2015, pp. 180-184.
- [8] C. Christidis and M. Devetsikiotis, "Blockchains and Smart Contracts for the Internet of Things," *IEEE Access*, vol. 4, pp. 2292-2303, 2016.

- [9] M. Sharples and J. Domingue, "The blockchain and kudos: A distributed system for educational record, reputation and reward," in *European Conference on Technology Enhanced Learning (EC-TEL)*, Springer, 2016, pp. 490-496.
- [10] G. Ateniese, B. Magri, D. Venturi, and E. Andrade, "Redactable Blockchain - or - Rewriting History in Bitcoin and Friends," in *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2017, pp. 111-126.
- [11] P. Voigt and A. Von dem Bussche, "The EU General Data Protection Regulation (GDPR): A Practical Guide," Springer International Publishing, 2017.
- [12] A. Grech and A. F. Camilleri, "Blockchain in education," *JRC Science for Policy Report*, vol. 10, p. 132, 2017.
- [13] X. Liang, S. Shetty, D. Tosh, C. Kamhoua, K. Kwiat, and L. Njilla, "ProvChain: A blockchain-based data provenance architecture in cloud environment with enhanced privacy and availability," in *Proc. IEEE/ACM CCGRID*, 2017, pp. 468-477.
- [14] A. Ramachandran and M. Kantarcioglu, "Using Blockchain and smart contracts for secure data provenance management," *arXiv preprint arXiv:1709.10000*, 2017.
- [15] M. Turkanović, M. Hölbl, K. Košič, M. Heričko, and A. Kamišalić, "EduCTX: A blockchain-based higher education credit platform," *IEEE Access*, vol. 6, pp. 5112-5127, 2018.
- [16] Z. Zheng et al., "Blockchain challenges and opportunities: A survey," *Int. J. Web Grid Serv.*, vol. 14, no. 4, pp. 352-375, 2018.
- [17] E. Androulaki et al., "Hyperledger fabric: a distributed operating system for permissioned blockchains," in *Proc. EuroSys*, 2018, pp. 1-15.
- [18] A. Reyna et al., "On blockchain and its integration with IoT. Challenges and opportunities," *Future Generation Computer Systems*, vol. 88, pp. 173-190, 2018.
- [19] G. Chen, B. Xu, M. Lu, and N.-S. Chen, "Exploring blockchain technology and its potential applications for education," *Smart Learning Environments*, vol. 5, no. 1, pp. 1-15, 2018.
- [20] R. Roman et al., "Mobile Edge Computing, Fog et al.: A Survey and Analysis of Security Threats and Challenges," *Future Generation Computer Systems*, vol. 78, pp. 680-698, 2018.
- [21] E. Politou, F. Casino, E. Alepis, and C. Patsakis, "Blockchain and GDPR: The right to be forgotten in the ledger context," *arXiv preprint arXiv:1810.02254*, 2018.
- [22] M. S. Ali et al., "Applications of blockchains in the Internet of Things: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1676-1717, 2019.
- [23] European Data Protection Board, "Guidelines 5/2019 on the criteria of the Right to be Forgotten in the search engines cases under the GDPR," 2019.
- [24] D. Boneh and V. Shoup, *A Graduate Course in Applied Cryptography*, Version 0.5, 2020.
- [25] S. Ruj et al., "Blockchain and the Right to be Forgotten," *IEEE Computer*, vol. 54, no. 1, pp. 54-63, 2021.